

GNU Bare-Metal Targeted Tools for Arm 64-bit Embedded Processors

NXP GCC for Arm Release 15-May-2020

Release description

Current release is based on:

- gcc 9.2.0
- binutils 2.32
- newlib 3.1.0
- gdb 8.1.1

This distribution provides support for newlib and newlib-nano.

This distribution provides tools for 32-bit host system. For 64 bit system, 32 bit libraries are required to run the tools.

Version identification

The version information (emitted via `--version` or `-v`) output format has changed to precisely define a build **BLD** identifier as an integer value whose value will always increase with each subsequent release. Example:

```
$ aarch64-none-elf-gcc --version
aarch64-none-elf-gcc (BLD = 1649) 9.2.0 20190812 (build.sh rev=gaf57174 s=F920 \
-i /opt/freescale Earmv8 -V release_gaf57174_build_Fed_Earmv8)
```

```
$ aarch64-none-elf-gcc -v
<snip>
gcc version 9.2.0 20190812 (build.sh rev=gaf57174 s=F920 -i /opt/freescale \
Earmv8 -V release_gaf57174_build_Fed_Earmv8) (BLD = 1649)
```

C Libraries usage

This toolchain is released with two prebuilt C libraries based on newlib: one is the standard newlib and the other is newlib-nano for code size. To distinguish them, we rename the size optimized libraries as:

libc.a --> libc_nano.a

libg.a --> libg_nano.a

To use newlib-nano, users should provide additional gcc link time option:

`--specs=nano.specs`

Nano.specs also handles two additional gcc libraries: `libstdc++_s.a` and `libsupc++_s.a`, which are optimized for code size.

For example:

```
$ aarch64-none-elf-gcc src.c --specs=nano.specs ${OTHER_OPTIONS}
```

This option can also work together with other specs options like

`--specs=rdimon.specs`

Please note that `--specs=nano.specs` is a linker option. Be sure to include it in linker options if compiling and linking separately.

Additional newlib-nano libraries usage

Newlib-nano is different from newlib in addition to the libraries' name. Formatted input/output of floating-point number are implemented as weak symbol. If you want to use %f, you have to pull in the symbol by explicitly specifying "-u" command option.

```
-u _scanf_float
```

```
-u _printf_float
```

Users can choose to use or not use semihosting by following instructions.

Semihosting

If you need semihosting, linking like:

```
$ aarch64-none-elf-gcc --specs=rdimon.specs ${OTHER_LINK_OPTIONS}
```

Semihosting v2

Semihosting v2 changes are documented in [specification](#).

The major change is the addition of an extensions mechanism to add more extensions in the future.

Non-semihosting/retarget

If you are using retarget, linking like:

```
$ aarch64-none-elf-gcc --specs=nosys.specs ${OTHER_LINK_OPTIONS}
```

User must provide its own IO subroutines (like _write, _read etc.) implementation otherwise warnings will appear on link stage:

```
undefined reference to `_write'
```

GCC Section Pragmas

GCC supports #pragma GCC section (text|data|bss|rodata) ".name" pragma.

This pragma allows to redefine standard sections by user defined section

```
#pragma GCC section (text|data|bss|rodata) ".name"
```

This pragma restores standard section

```
#pragma GCC section (text|data|bss|rodata) "default"
```

Fore example:

```
#pragma GCC section bss ".foobss"
#pragma GCC section data ".foodata"
#pragma GCC section rodata ".foorodata"
#pragma GCC section text ".footext"
char c1;
char c2 = 2;
const char c3 = 3;
int f1 (int x) { return x; }
#pragma GCC section bss "default"
#pragma GCC section data "default"
```

```
#pragma GCC section rodata "default"
#pragma GCC section text "default"
```

Installing executables on Windows

Some executables built by MinGW requires additional libraries. Thus to support out-of-the-box execution, libwinpthread.dll comes with toolchain. This library can be moved in appropriate place if host system requires. Or MinGW package can be installed to get executables working.

Installing executables on Linux

"32-bit compatibility libraries" should be installed just in order to run 32-bit toolchains:

Ubuntu 14

```
$ sudo apt-get install lib32z1 lib32ncurses5 lib32bz2-1.0
```

Ubuntu 16

```
$ sudo dpkg --add-architecture i386
$ sudo apt-get update
$ sudo apt-get install libc6:i386 libncurses5:i386 libstdc++6:i386 lib32z1
```

Debian

```
$ sudo apt-get install lib32z1 lib32ncurses5 lib32stdc++6
```

CentOS

```
$ sudo yum install glibc.i686 ncurses-devel.i686
```

GDB with Python

Additionally if you want to use gdb python build (aarch64-none-elf-gdb-py) on Linux, you need to install 32 bit python2.7 and compatibility libraries.

Ubuntu

```
$ sudo apt-get install libpython2.7:i386
$ sudo apt-get install python2.7-minimal:i386 python2.7:i386 python2.7-dev:i386
```

And after that switch default python to new one:

```
$ sudo ln -s usr/bin/python2.7 python
```

CentOS

```
$ sudo yum install python-libs.i686
```

Windows

To use gdb python build (aarch64-none-elf-gdb-py.exe) on Windows, you need to install 32 bit python2.7 no matter 32 or 64 bit Windows. Please get the package from <https://www.python.org/download/>.

Release history

Date	Description
29-January-2020	Initial release
bld=1649 rev=gaf57174	No changes